

Documentation : Authentification JWT (Clean Architecture)

L'objectif de cette implémentation est de fournir un système de sécurité complet et robuste via des jetons **JWT (JSON Web Tokens)**. Cela permet aux traiteurs de s'inscrire (avec un mot de passe crypté) et de se connecter pour obtenir un token d'accès nécessaire pour les routes protégées.

Architecture & Implémentation

1. La Couche Domaine (**domain/**)

- **Rôle** : Objets purs pour gérer l'authentification sans dépendre de Spring Security.
- **AuthCredentials.java** : Représente l'email et le mot de passe non crypté pour la connexion.
- **JwtToken.java** : Représente le token JWT final généré.

2. La Couche Application (**application/**)

- **Rôle** : Les règles métier de connexion et le hachage.
- **LoginUseCase.java** : Port entrant pour se connecter.
- **PasswordEncoderPort.java** : Port sortant (interface) pour crypter les mots de passe.
- **JwtServicePort.java** : Port sortant (interface) pour générer le JWT.
- **AuthApplicationService.java** : Service métier qui vérifie l'email/mot de passe, s'assure que le compte est actif, et renvoie le JWT.
- **CatererApplicationService.java** : Modifié pour crypter le mot de passe du traiteur lors de son inscription via le **PasswordEncoderPort**.

3. La Couche Infrastructure (**infrastructure/security/**)

- **Rôle** : L'implémentation technique avec Spring Security et la librairie JJWT.
- **BCryptPasswordEncoderAdapter.java** : Implémente le hachage via Spring BCrypt.
- **JwtTokenProviderAdapter.java** : Implémente la création et lecture du JWT avec la librairie **jjwt**.
- **JwtAuthenticationFilter.java** : Filtre Spring qui lit le token dans l'en-tête HTTP **Authorization: Bearer**
- **SecurityConfig.java** :
 - Configuration de l'application en mode **STATELESS** (sans session de serveur).
 - Enregistrement du **JwtAuthenticationFilter**.
 - Ouverture publique des routes **/api/caterers/**** (Création) et **/api/auth/**** (Login).

4. La Couche Présentation (**presentation/**)

- **Rôle** : Recevoir les appels HTTP liés à la connexion.
- **AuthResource.java** : Expose la route **POST /api/auth/login**.
- **LoginRequestDTO.java** : DTO (Data Transfer Object) pour recevoir l'email et le mot de passe depuis le client (ex: Postman ou Angular).

Comment Tester (Flow de Connexion)

[!CAUTION] Les mots de passe sont désormais cryptés avec BCrypt en base de données. Les anciens comptes créés manuellement ou avant cette mise à jour (avec des mots de passe en texte clair) ne fonctionneront plus.

1. Création d'un compte (Inscription) :

- Requête : `POST /api/caterers`
- Le mot de passe envoyé sera automatiquement haché avant d'être sauvegardé dans MySQL.

2. Connexion (Login) :

- Requête : `POST /api/auth/login`
- Body JSON : `{"email": "votre@email.com", "password": "votre_mot_de_passe"}`
- **Réponse attendue** : Un token JWT valide sous la forme `{"token": "eyJhbGciOiJIUzI1NiJ9..."}`.

3. Accès protégé :

- Requête : `GET /api/caterers/email/votre@email.com`
- **Important** : Dans Postman, vous devez ajouter un Header HTTP.
- Key : `Authorization`
- Value : `Bearer <votre_token_jwt>`
- Sans cela, l'API renverra une erreur `403 Forbidden`.